Readable & Writable Streams

Why it matters

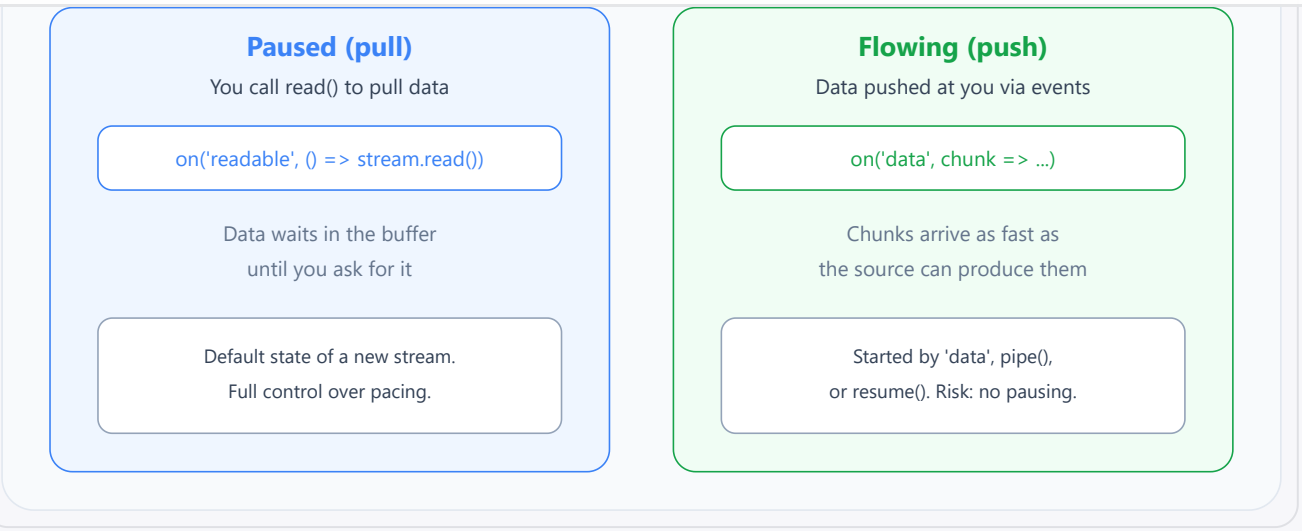
Readable and Writable are the two halves of every data movement in Node. The request coming into your server is a Readable; the response going out is a Writable. The file you read is a Readable; the file you write is a Writable. Duplex and Transform — the subjects of the next topic — are literally built by combining these two. If you are shaky on how a Readable delivers data and how a Writable accepts it, everything above them (piping, compression, HTTP bodies) is guesswork.

The part that trips people up, and the part interviewers probe, is that a Readable can operate in two completely different **modes** — paused and flowing — and choosing the wrong one, or accidentally switching between them, causes two of the most common stream bugs in production: silently lost data, and memory blowups from ignored backpressure. Getting this right is the difference between a service that streams a million-row export cleanly and one that OOMs at row 200,000.

The concept, simply

A **Readable** is a faucet. A **Writable** is a drain. Your job is to get water from the faucet to the drain without overflowing the sink in between — that sink is the internal buffer, and not overflowing it is backpressure.

The subtlety is entirely on the Readable side, because a faucet can deliver water two ways. In **paused mode** (also called pull mode), the data sits in the stream's buffer and nothing happens until *you* call `read()` to pull a chunk. You are in control of the pace. In **flowing mode** (push mode), the stream fires `'data'` events at you as fast as the source can produce, and you must keep up or handle backpressure yourself.



In paused mode you pull chunks by calling read(); in flowing mode chunks are pushed to you through data events until you pause or the source ends.

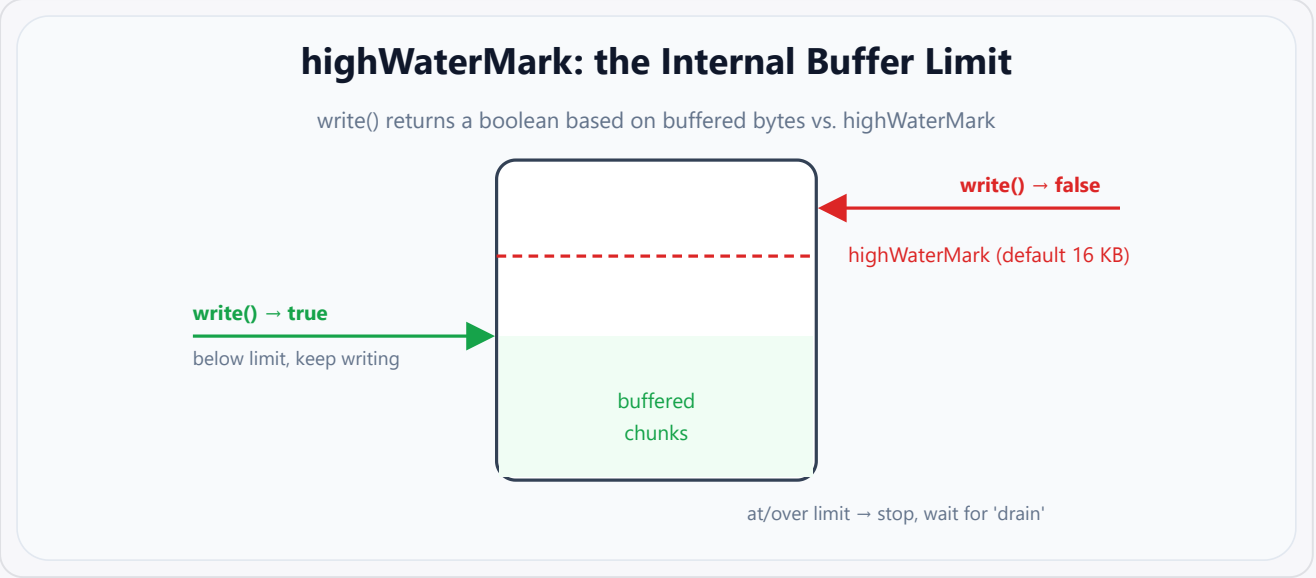
A brand-new Readable starts **paused**. It switches to **flowing** the moment you do any of three things: attach a `'data'` listener, call `.pipe()`, or call `.resume()`. It switches back to paused if you call `.pause()` (and, when piped, whenever the destination signals backpressure). The single most important practical rule follows from this: **attaching a `'data'` listener starts the flow immediately.** If you attach it and then do async setup before you are ready to handle chunks, the chunks that arrive in the meantime are gone.

How it works

Reading, in detail. A Readable pulls from its underlying source (a file descriptor, a socket) into its internal buffer via an internal `_read()` method. It keeps pulling until the buffered amount reaches the `highWaterMark` (default 64 KB for files), then stops asking for more. In paused mode, each `read(n)` you call removes up to `n` bytes from that buffer and returns them (or `null` if not enough is buffered yet); as the buffer drains below the mark, the stream resumes pulling from the source. In flowing mode the same buffering happens, but the stream automatically drains its buffer by emitting `'data'` events — which is exactly why flowing mode without piping is dangerous: there is no automatic pause if your `'data'` handler is slower than the source.

Writing, in detail. A Writable exposes `write(chunk)`, which appends to its own internal buffer and asks the underlying resource to consume it via `_write()`. The return value is the backpressure signal: `true` means the buffered bytes are below the

does not error; it just keeps piling data into the buffer, and memory grows without bound. That is the classic slow-disk / fast-source memory leak.



write() returns true while buffered bytes are below the highWaterMark and false once they reach it; on false you must wait for drain before writing again.

Ending. You signal a Writable is done with `end([finalChunk])`. Once all buffered data is flushed to the underlying resource, it emits `'finish'`. After the resource itself is released it emits `'close'`. On a Readable, exhaustion is signalled by `'end'`. These are distinct events for distinct streams, and confusing `'end'` (readable is done producing) with `'finish'` (writable is done consuming) is a common interview slip.

Async iteration. Modern Node lets you consume a Readable with `for await ... of`. This is paused-mode consumption in disguise — the loop pulls one chunk, awaits your body, then pulls the next — so backpressure is automatic and you get clean `try/catch` error handling. It is the most readable way to consume a stream when you need per-chunk logic.

Code examples

1. Consume with `for await` (recommended default). Backpressure and errors are handled by the language, not by hand.

JAVASCRIPT Copy

```
async function countLines(path) {
  let count = 0, tail = '';
  const stream = createReadStream(path, { encoding: 'utf8' });
  for await (const chunk of stream) {   // pulls one chunk at a time
    const parts = (tail + chunk).split('
');
    tail = parts.pop();                 // last piece may be incomplete
    count += parts.length;
  }
  if (tail) count++;                     // final line with no newline
  return count;
}
console.log(await countLines('access.log'));
```

2. Respecting Writable backpressure by hand. The pattern `pipeline()` automates — worth writing once so you understand what it does.

JAVASCRIPT Copy

```
import { createWriteStream } from 'node:fs';
import { once } from 'node:events';

async function writeLots(path, n) {
  const out = createWriteStream(path);
  for (let i = 0; i < n; i++) {
    const ok = out.write(`row ${i}
`);
    if (!ok) {
      // Buffer is full — pause until it drains below highWaterMark.
      await once(out, 'drain');
    }
  }
  out.end();
  await once(out, 'finish');   // all data flushed to disk
}
await writeLots('rows.txt', 1_000_000);
```

3. Creating your own Readable. Push data with `push(chunk)`; signal the end with `push(null)`.

```
import { Readable } from 'node:stream';

// Simplest: from an iterable / generator.
const numbers = Readable.from((function* () {
  for (let i = 1; i ≤ 5; i++) yield `${i}
`;
})());

// Explicit form: implement _read and push chunks yourself.
class Counter extends Readable {
  #n = 0;
  constructor(max) { super(); this.max = max; }
  _read() {
    this.#n++;
    if (this.#n > this.max) this.push(null);        // end of stream
    else this.push(Buffer.from(`${this.#n}
`));   // emit a chunk
  }
}
new Counter(3).pipe(process.stdout);   // prints 1
2
3
```

4. Creating your own Writable. Implement `_write`; call the callback when the chunk is handled (this is how backpressure is reported upstream).

JAVASCRIPT Copy

```
import { Writable } from 'node:stream';

class JsonlLogger extends Writable {
  constructor() { super({ objectMode: true }); }   // accept objects, not
  _write(record, _enc, cb) {
    process.stdout.write(JSON.stringify(record) + '
');
    cb();   // MUST call cb — err first if it failed. Until then, backpres
  }
}
const log = new JsonlLogger();
log.write({ level: 'info', msg: 'up' });
```

Common mistakes & gotchas

- **Attaching `'data'` too early.** It flips the stream to flowing immediately. If you then `await` something before processing, chunks emitted during the await are lost. Attach the listener only when you are ready to handle chunks, or use `for await`.
- **Ignoring the return value of `write()`.** If it returns `false` and you keep writing, the Writable's buffer grows without bound — unbounded memory, not an error. Wait for `'drain'`.
- **Mixing paused and flowing consumption.** Calling `read()` while also having a `'data'` listener, or piping a stream you are also manually reading, leads to data going to one consumer and not the other.
- **Confusing `'end'` with `'finish'`.** `'end'` = a Readable has no more data to give. `'finish'` = a Writable has flushed everything you gave it. Different streams, different meanings.
- **Forgetting to call the `_write` callback** in a custom Writable. Until you call `cb()`, the stream thinks the chunk is still being processed and applies backpressure forever — the pipeline silently stalls.
- **Calling `push()` after `push(null)`** in a custom Readable, or pushing faster than `_read` is called — both throw or leak.

Best practices

- **Prefer `for await ... of`** for consuming a Readable when you need custom per-chunk logic — it gives you automatic backpressure and normal `try/catch`.
- **Prefer `pipeline()`** when you are just moving data from a Readable to a Writable (or through Transforms) — do not hand-roll the drain loop unless you are learning or have a special case.
- **When building a custom stream, use the constructor options** (`read`, `write`, `transform` functions) or subclass and implement the `_`-prefixed methods — and always call the provided callback.

- Use `Readable.from(iterable)` to turn an array, generator, or async iterator into a stream instead of implementing `_read` by hand.

⭐ Interview lens

Q: What are the two modes of a Readable stream, and what triggers each?
Paused (pull) and flowing (push). A new stream starts paused. It goes flowing when you add a `'data'` listener, call `.pipe()`, or call `.resume()`; it returns to paused on `.pause()` or, when piped, when the destination applies backpressure. In paused mode you pull with `read()`; in flowing mode chunks arrive via `'data'` events.

Q: You attach a `'data'` handler but some chunks seem to vanish. Why? (Trap.)
Attaching `'data'` switches the stream to flowing mode *immediately*. If there is any asynchronous gap between attaching the listener and actually being ready — or if a second consumer is added later — chunks emitted in that window are dropped because flowing mode does not wait. Fix: use `for await`, or don't start the flow until ready.

Q: What does `stream.write()` returning `false` mean, and what should you do?
The Writable's internal buffer has reached the highWaterMark — backpressure. Stop calling `write()` and wait for the `'drain'` event before resuming. Ignoring it doesn't throw; it just grows memory unboundedly.

Q: Difference between the `'end'` and `'finish'` events?
`'end'` fires on a Readable when there is no more data to read. `'finish'` fires on a Writable after `end()` is called and all buffered data has been flushed to the

Q: In a custom Writable's `_write(chunk, enc, cb)`, what happens if you never call `cb`?

The stream believes the chunk is still being processed, so it never requests the next one and applies backpressure indefinitely — the pipeline stalls silently with no error. You must call `cb()` on success or `cb(err)` on failure.

Q: How do you signal the end of data from inside a custom Readable?

Call `this.push(null)`. Every earlier `push(chunk)` emits data; `push(null)` marks EOF and eventually triggers the `'end'` event downstream.

Cheat-card

- **Readable** = source (faucet). Starts **paused**; goes **flowing** on `data` listener / `pipe()` / `resume()`.
- **Paused**: pull with `read()` on `'readable'`. **Flowing**: pushed via `'data'`.
- **Writable** = sink (drain). `write()` → `false` means buffer ≥ highWaterMark; wait for `'drain'`.
- **End signals**: Readable emits `'end'`; Writable `end()` → `'finish'` → `'close'`.
- **Build**: custom Readable `push(chunk)` / `push(null)`; custom Writable `_write(chunk,enc,cb)` — *always call cb*.
- **Consume**: `for await ... of` for logic; `pipeline()` to just move bytes. `objectMode` for records.

QUICK CHECK — no marks, just for you.

1. What triggers a new (paused) Readable stream to switch into flowing mode?

- ◯ Calling .read() once to pull the first chunk

- [ ] Attaching a "data" listener, calling .pipe(), or calling .resume()
- [ ] Setting objectMode: true in the constructor

Check answer

2. You attach a "data" handler but some chunks seem to vanish. What is the likely cause?

- [ ] The chunks were larger than the highWaterMark and silently discarded
- [ ] Node deduplicates identical chunks automatically
- [ ] The stream ended before the "data" listener was registered, which is impossible to fix
- [ ] Attaching "data" switches the stream to flowing mode immediately, so chunks emitted during an async gap before you are ready are dropped

Check answer

3. What is the difference between the "end" and "finish" events?

- [ ] "end" fires on a Readable when there is no more data to read; "finish" fires on a Writable after end() is called and all buffered data has been flushed
- [ ] "end" and "finish" are aliases that fire on every stream simultaneously
- [ ] "finish" fires on a Readable; "end" fires on a Writable
- [ ] "end" fires when the file descriptor closes; "finish" fires when it opens

Check answer

Previous
Streams Overview

Next
Transform & Duplex Streams

LEARN MORE

ARTICLES & DOCS

- Stream | Node.js Documentation
  nodejs.org
- Node.js Streams: Everything you need to know
  freecodecamp.org

VIDEOS



Learn Node.js Streams in 25 minutes | NodeJS Tutorials for Beginners
Dipesh Malvia

Hand-picked references for this lesson.